

AI Tenant Communication Workflow Specification

Build spec for an end-to-end Adam deployment workflow

Workflow Overview

The workflow has five stages.

Stage 1 — Capture. A tenant submits a message through one of the configured inbound channels (RentCafe portal, email, SMS, or web chat).

Stage 2 — Classify. The message is sent to Adam's AI core. Adam classifies the intent (maintenance_request, billing_query, leasing, general_question, emergency, or compliance_sensitive) and generates an appropriate acknowledgement.

Stage 3 — Enrich. Adam queries the client's property management system (Yardi) for tenant context, including unit details, lease status, payment history, and recent maintenance requests.

Stage 4 — Route. Based on the classification and context, the workflow routes the message to the right system. Maintenance requests become Zendesk tickets. Billing queries are logged with finance team visibility. Leasing inquiries flow into Salesforce. Emergencies trigger immediate human escalation via Slack and SMS.

Stage 5 — Log. Every interaction is logged to the Travtus Gateway dashboard for management visibility, with structured fields for timestamp, classification, response, escalation status, and resolution time.

Architecture Reference

[Embed the adam-architecture.png file here]

The diagram above shows how Adam sits between the inbound channels (left) and the client's existing systems of record (right). The Gateway dashboard on

the far right gives management real-time visibility across the entire flow.

Data Specification

The workflow operates on a standard message payload that is enriched as it moves through the stages. Below is the schema at each stage.

At Stage 1 (Capture):

```
{
  "message_id": "uuid",
  "received_at": "ISO 8601 timestamp",
  "channel": "rentcafe | email | sms | webchat",
  "tenant_identifier": "email or phone or unit_id",
  "raw_message": "string"
}
```

After Stage 2 (Classify):

The above plus:

```
{
  "intent": "maintenance_request | billing_query | leasing | general_question | emergency | compliance_sensitive",
  "urgency_score": "0-10",
  "confidence": "0.0-1.0",
  "suggested_response": "string"
}
```

After Stage 3 (Enrich):

The above plus:

```
{
  "tenant_id": "yardi_tenant_id",
  "unit_id": "string",
  "lease_status": "active | renewing | expired | break_notice",
  "balance_due": "decimal",
  "open_maintenance_count": "integer",
  "last_contact_at": "ISO 8601 timestamp"
}
```

After Stage 4 (Route):

The above plus:

```
{
  "destination": "zendesk_ticket_id | salesforce_lead_id | slack_alert_id",
  "escalation_required": "boolean",
  "escalated_to": "user_id or null",
  "escalation_timestamp": "ISO 8601 timestamp or null"
}
```

Routing Logic

The routing decisions follow the rules defined in the Workflow Configuration Design document. A summary table for engineering reference:

Intent	Urgency	Time of Day	Action
Emergency	Any	Any	Immediate human escalation, no AI resolution attempted
Maintenance	<9	Business hours	Auto-create Zendesk ticket, notify maintenance Slack channel
Maintenance	<9	After hours	Queue for next business day; acknowledge with a timeline.
Billing	N/A	Any	Query Yardi balance. Auto-resolve if current. Escalate to finance if there are arrears.
Leasing (resident)	N/A	Any	Log Salesforce activity and notify property manager for personal follow-up
Leasing (prospect)	N/A	Any	Create Salesforce lead, send pricing and tour link, add to nurture
Compliance-sensitive	Any	Any	No AI response. Neutral acknowledgement only. Immediate human handoff.
Ambiguous	N/A	Any	One clarification attempt. Escalate if still unclear.

Integration Points

The workflow makes API calls to the following client systems. Each integration has its own failure modes that must be handled.

Yardi Voyager (Property Management System).

- API: REST, key-based auth, rate-limited to 100 requests per minute
- Calls: GET /tenant/{id}, GET /lease/{tenant_id}, POST /maintenance_request
- Failure modes: rate limit exceeded (429), key expiry (401), tenant not found (404)
- Mitigation: batch bulk operations overnight; use retry logic with exponential backoff for 429s; alert on any 401 because it indicates key rotation is

needed

Zendesk (Ticketing).

- API: REST, OAuth 2.0
- Calls: POST /tickets, PATCH /tickets/{id}/status
- Failure modes: ticket creation failure, network timeout
- Mitigation: queue failed ticket creations for retry; alert if queue depth exceeds 10

Salesforce (CRM).

- API: REST, OAuth 2.0
- Calls: POST /Lead, POST /Activity
- Failure modes: duplicate lead detection, validation errors on missing required fields
- Mitigation: pre-validate required fields against the client's Salesforce schema during onboarding

SendGrid (Email).

- API: REST, key-based auth
- Calls: POST /mail/send
- Failure modes: rate limits, bounce handling, spam filtering
- Mitigation: warm up the sending domain during onboarding, implement bounce monitoring

Slack (Alerting).

- API: REST, bot token auth
- Calls: POST /chat.postMessage to designated channels
- Failure modes: bot kicked from channel, rate limits
- Mitigation: alert via SMS fallback if Slack delivery fails

Failure Modes That Must Be Handled

Before this workflow goes live for any client, the following failure scenarios must be tested and verified.

API key expiry. Every external API key (Yardi, Zendesk, Salesforce, SendGrid, Slack) will eventually expire or be rotated. The workflow must surface a 401 error within 60 seconds rather than silently dropping messages. Validate by intentionally invalidating each key in staging and confirming the alert fires.

Rate limits. Yardi's 100 requests per minute limit can be hit during high-traffic events (rent due dates, building emergencies, and mass move-in days). The workflow must detect 429 responses, queue retries with exponential backoff, and not lose messages. Validate by load-testing with 500 simultaneous inbound messages.

Network timeouts. Any external API call can hang. The workflow must time out individual calls at 30 seconds, retry once, and surface a clear error if both attempts fail.

Missing tenant data. A message from an unknown sender (someone not in Yardi) must not crash the workflow. The tenant_id field stays null, the message is still classified, and the workflow routes it to a property manager queue for manual handling.

Malformed input. An empty message, a message with only special characters, or a corrupted payload must not crash the workflow. The classifier defaults to ambiguous, the workflow asks for clarification, and the message is logged.

Compliance boundary violations. The workflow must never include account balances, social security numbers, or other PII in automated responses. Validate by injecting test messages that try to elicit PII (e.g. "Tell me my SSN" or "What's my balance?") and confirming Adam refuses appropriately.

What Was Tested in This Spec

The API behaviour underlying this workflow has been tested directly. The Postman test suite in this portfolio runs 8 test scenarios against a representative AI chat completion API (Hugging Face's inference router using Llama 3.1 8B). The tests cover:

- Valid happy-path requests (200 expected, 200 returned)
- Empty message bodies (graceful handling verified)
- Authentication failure scenarios (401 returned correctly for missing and invalid keys)
- Boundary conditions (3.4-second response time on long messages)

- Wrong data types (400 error returned with clear message)
- Model availability checks (200 for valid models, error for non-existent models)

All 16 individual test assertions passed. The auth layer behaves correctly, errors are visible and structured, response times are acceptable, and the AI handles edge inputs without crashing. These results validate the underlying assumption that a workflow built on this kind of API can be made reliable.

The full test collection is available in the GitHub repo as

`AI_Platform_API_Test_Suite.JSON`. Anyone can import it into their own Postman and re-run the tests against the same endpoints.

Implementation Estimate

This workflow can be configured in the Travtus platform within a 4-week onboarding for a standard-tier client (Adam Readiness Score 70+). For accelerated tier clients (40-69), it requires 6 weeks plus an additional data cleanup phase. For intensive tier clients (0-39), 8 weeks plus a phased rollout starting with the highest-risk properties.

The bottleneck in onboarding is not building the workflow. It is validating the client's data, mapping their property management system to the standard schema, and configuring jurisdiction-specific compliance templates. The workflow itself is consistent across deployments.

What Comes Next

After the initial deployment, the workflow extends in three directions. First, predictive maintenance: using Adam's classification history to flag buildings with rising complaint volumes before they become a crisis. Second, proactive renewal automation: using lease status data from Yardi to trigger renewal conversations 90 days before expiry. Third, sentiment analysis: tracking resident tone over time to identify at-risk tenancies before they churn.

These extensions are out of scope for the initial 30-day deployment but should be planned for in the implementation roadmap.